```
LOCAL MODELS IN PRACTICE
running a 4-bit 8B on a real, hard task — and what actually moved
text-to-SQL · execution-verified · 16 GB laptop · MLX
```

*A personal field report from hands-on local-model experiments on constrained hardware. One model studied deeply (Qwen3-8B, 4-bit) on a real, hard task (text-to-SQL), against a frontier baseline, across ~10 different methods. The absolute numbers are one data point; the **mechanisms and the setup lessons generalize**.*

TL;DR (the quotable part)

1. **For local-model* usability*, scaffolding beats parameters. A 4-bit 8B* with a tool* — let it run its own SQL, see the error/result, and fix it — reached its accuracy ceiling **gold-free in ~2 turns**, beating sampling, voting, and fine-tuning. The highest-leverage decision isn't the model or the quant level; it's **whether the model gets tools and a feedback loop.**
2. **There is a hard capability ceiling, and it belongs to the base model.** On hard, heterogeneous queries the 4-bit 8B topped out at ~40% *across every method tried*. Raising that needs a bigger/better base model or RL — not prompt or scaffold tricks.
3. **Accuracy is not one number — it depends on task* shape.* On homogeneous tasks (one skill, many instances) a local model is strong and can even self-improve to ~98%. On heterogeneous tasks (every query different) it is capability-bound. Any eval reporting a single accuracy hides this.
4. **4-bit quantization is the practical small-RAM lever.** Everything ran in 4-bit at ~5–6 GB resident on a 16 GB machine. Weight-*streaming* from SSD is the wrong lever for interactive/tool use (latency) — quantize instead.

Setup (what I actually ran)

- **Hardware:** Apple Silicon laptop, **16 GB unified memory** — deliberately small; a good proxy for the

"limited RAM" question.

- **Runtime:** MLX (Apple's local inference stack). One model resident at a time.
- **Local model:** `Qwen3-8B`, **4-bit** quantized (`mlx-community/Qwen3-8B-4bit`), ~5–6 GB resident.
- **Frontier baseline:** a frontier model (Claude) via CLI — used as a top-of-line reference and as a

"teacher" to generate correct examples for fine-tuning experiments.

- **Task: BIRD** mini-dev, **text-to-SQL** — natural-language question + database schema → SQL. Real

SQLite databases. Used the *simple+moderate* difficulty slice; held-out set of **40** questions, fixed across every method for apples-to-apples comparison.

- **Eval function (the core of it): execution accuracy** — run the model's SQL *and* the gold SQL

against the real database and compare result sets. Free, deterministic, fully automated. No human grading, no LLM-as-judge.

Results at a glance

```
ACCURACY ON HELD-OUT (n=40)          bar = % of questions correct (40 cols = 100%)

frontier baseline (Claude)   ██████████████████████████████░░░░░░░░░░   75%   (measured, n=40)
8B + TOOL (run SQL → fix)    ████████████████░░░░░░░░░░░░░░░░░░░░░░░░    40%   ★ gold-free, ~2 turns
8B oracle best-of-8          ████████████████░░░░░░░░░░░░░░░░░░░░░░░░    40%   (needs the answer key)
8B self-critique loop (FT)   ███████████████░░░░░░░░░░░░░░░░░░░░░░░░░    38%
8B + tool + rich schema      ███████████████░░░░░░░░░░░░░░░░░░░░░░░░░    37%
8B one-shot (greedy)         █████████████░░░░░░░░░░░░░░░░░░░░░░░░░░░    33%
8B self-consistency (vote)   █████████████░░░░░░░░░░░░░░░░░░░░░░░░░░░    33%
8B fine-tuned on traces      ▒▒▒▒▒▒▒▒▒▒▒▒▒░░░░░░░░░░░░░░░░░░░░░░░░░░░   ~flat (no gain)

                              └──────────── the ~40% wall ───────────┘
```

The whole story in one picture: every lever — sampling, voting, fine-tuning, tools, schema enrichment — lands at **~40%**. Tool-use is the only one that gets there *cheaply and without the answer key*.

Every method I tried (the full list)

#	Method	Idea	Result (n=40)
1	One-shot (greedy)	single attempt, plain prompt	13/40 (33%)
2	Self-consistency	8 samples, majority result wins	13/40 (33%)
3	Oracle best-of-8	8 samples, keep if *any* is correct (upper bound)	16/40 (40%)
4	pass@k curve	how the oracle bound grows with k	plateaus at 40%
5	**Agentic tool-use**	run the SQL, read error/result, fix; gold-free stop	**16/40 (40%) ★**
6	Self-distillation	fine-tune on the model's own verified-correct traces	flat
7	Teacher-bootstrap distill	fine-tune on 50 *frontier-correct* traces	flat
8	Teacher-bootstrap, scaled	same, scaled to ~100 correct traces	flat
9	Factored / decomposed distill	teach it to break a query into sub-skills, then compose	flat
10	Self-critique loop (FT)	fine-tune the DRAFT→CHECK→FINAL loop into the weights	15/40 (~38%)
11	Tool + rich schema	give sample values per column (value-format knowledge)	15/40 (37%)

Methods 6–9 are the "can a small model fine-tune itself good at this?" line of attack. **They all stayed flat** — even on correct frontier-authored examples — because ~50–100 examples can't cover a heterogeneous domain, and the bottleneck is generation capability, not training signal.

What the model actually gets wrong — and how the tool fixes it

Real failure mode (representative). The schema hides a foreign key the model forgets to join:

```
-- SCHEMA (compact form given to the model)
superhero(id, superhero_name, full_name, gender_id, race_id, publisher_id, ...)
gender(id, gender)

-- QUESTION : "How many superheroes are female?"
-- HINT     : female refers to gender = 'Female'

-- 8B one-shot, turn 1                                          ✗ WRONG
SELECT COUNT(*) FROM superhero WHERE gender = 'Female';
   └→ sqlite error: no such column: gender

-- 8B with the run_sql tool, turn 2 (after seeing the error)    ✓ CORRECT
SELECT COUNT(*)
FROM   superhero AS T1
JOIN   gender    AS T2 ON T1.gender_id = T2.id
WHERE  T2.gender = 'Female';
   └→ runs, result matches gold
```

That single feedback step — "*no such column: gender*" — is information the model didn't have at generation time. That is the whole reason tool-use works where blind voting doesn't.

The internalized version (method 10) teaches the model to do that critique *itself*, without the tool:

```
DRAFT:  SELECT COUNT(*) FROM superhero WHERE gender = 'Female';
CHECK:  'gender' is not a column on superhero — it's gender_id, a foreign key into
        gender(id, gender). This needs a join.
FINAL:  SELECT COUNT(*) FROM superhero T1
        JOIN gender T2 ON T1.gender_id = T2.id
        WHERE T2.gender = 'Female';
```

It reaches ~38% tool-free — the loop *is* partly learnable into the weights — but still can't beat the ~40% ceiling.

Why richer schema (method 11) didn't help. Sample values were injected inline so the model could see the value formats:

```
plain schema:   gender(id, gender)
rich schema:    gender(id, gender[Male,Female])      ← sample distinct values inline
```

It fixed *some* value-format mistakes but the overall number didn't move — schema ignorance wasn't the binding constraint; raw generation capability was.

The capability ceiling, seen as a curve

```
pass@k  —  oracle "any-of-k samples correct"  (upper bound with a perfect selector)

 45% |
 40% |                  • — • — • — •      ← PLATEAU: more samples add nothing
 35% |          •
 30% |     •
 27% |  •
     +----+----+----+----+----+----+----+
        1    2    4    6    8   12   16        k = number of samples
```

Past k=8 the bound stops rising: on ~60% of hard queries the correct SQL **never appears in any sample**. That is a generation-capability limit, not a selection or sampling limit. No local trick conjures an answer the model can't produce.

The five findings, with the "why"

1. Tools > parameters for usability. Execution feedback is *real information the model lacks*. Plain voting can't replicate it because the model's wrong answers don't agree on one wrong result, so the majority is often wrong. Budget for an agent loop + a verifier, not just a bigger model.

2. The ~40% ceiling is base capability, confirmed six ways (sampling, voting, two kinds of fine-tuning, tool-use, schema enrichment all cap there). Scaffolding sets how *close* you get to the ceiling; it doesn't raise it. Raising it needs a stronger base model, far larger fine-tuning corpora (published SQL wins use ~900k synthetic examples), or RL.

3. Task shape _dominates the number._ Same model, same effort: ~98% on a homogeneous self-taught skill, ~40% on heterogeneous BIRD. A single "accuracy" per model is misleading without a task-shape axis.

4. Locally you can teach a loop_, not broad_ coverage_. Fine-tuning on correct traces (even frontier-correct) left heterogeneous accuracy flat; what transferred was the self-critique_ behavior (~38%). You can cheaply install a reusable habit; you can't cheaply install domain breadth.

5. Quantization is the right small-RAM lever; streaming is not. 4-bit Qwen3-8B ran fine at ~5–6 GB and was capability-bound, not quant-bound, on this task. SSD weight-streaming adds per-token latency that kills an interactive tool loop — and the loop is where the value is.

Practical gotchas (the time-savers)

```
⚠  Eval truncation = false failures.
   If eval max_tokens < the model's answer length, the answer is silently chopped
   and scored 0. This produced a "total failure" that was actually 98%. Always
   diagnose model-vs-eval before believing a bad score.

⚠  One model at a time on small RAM.
   16 GB holds one ~8B comfortably; don't co-load. Fine-tuning needs aggressive
   settings (batch 1, fewer layers, short sequences, gradient checkpointing) or
   it OOMs on long inputs.

⚠  Compact the prompt.
   table(col, col, …) instead of full CREATE DDL was ~1.5× faster, no accuracy loss.
   Prompt size is a real per-call cost.

⚠  Progress bars corrupt output.
   Library download/progress bars merge into stdout and break parsed results —
   disable them in batch runs.

✓  Use a free, deterministic verifier when one exists (execution, unit tests, regex).
   It removes humans and LLM-judges from the loop and makes the eval trustworthy.
```

Honest caveats

- **One model, one benchmark, one difficulty slice, n=40.** The ~40% number is specific to Qwen3-8B-4bit on

hard BIRD. The *mechanisms* (tools reach the ceiling, the ceiling is base capability, shape matters, quant works) are what generalize.

- This is **one deep data point**, not a model leaderboard. A matrix across other open-weight models is the

obvious next step — the harness is model-agnostic, so it's mostly config.

Recommendations

1. **Evaluate by task complexity — and add two axes:** *task homogeneity* (homogeneous vs heterogeneous) and *with-tools vs without-tools*. They explain more variance than model choice.
2. **Make "agent loop + free verifier" a first-class condition**, not an afterthought. It's the difference between a leaderboard and a useful conclusion.
3. **Quantize, don't stream.** Test 8-bit and 4-bit; expect 4-bit to be the practical floor. Treat SSD-streaming as a curiosity, not an interactive-deployment path.

4. **The headline is "scaffolding beats parameters for local usability."** Non-obvious, defensible, and far more interesting than "model X scored Y%".
5. **Build the harness once.** An execution-verified, difficulty-sliced harness with a frontier baseline and a 4-bit setup pays for itself; adding more candidate models is then a small delta.

Next step would be extending the harness to other open-weight models (e.g. GLM-5.2 / Gemma / DeepSeek) to produce a full model × quant × (with/without tools) matrix.

Addendum (update): interactive tools cross the 40% wall (confirmed, n=80)

A later experiment supersedes the "~40% ceiling across every lever" finding above. That ceiling held only for **static context** (one-shot, sampling, fine-tuning, static rich-schema). Letting the model **interactively explore** the database before committing — foreign-key join paths, on-demand sample values, then run-and-fix — crosses it.

```
CONFIRMATION (fresh held-out, n=80, zero salary)

one-shot                  ███████████████░░░░░░░░░░░░░░░░░░░░░░░░░   37%
interactive tools         ███████████████████░░░░░░░░░░░░░░░░░░░░░   48%   +11 points, lead held throughout
                          (foreign keys + on-demand sample values + run/fix)
```

The +11 points is about 2 standard deviations at n=80 (robust, not small-sample noise). One caveat learned the hard way: a **richer** toolset (adding fuzzy-find and exploratory-subquery tools, multi-round) *lowered* accuracy on the weak model (distraction). The lesson is that a small model's toolspace must be **verified-selected — keep only tools that measurably raise accuracy, prune the rest — not expanded.**

Net: this reinforces the report's headline ("scaffolding beats parameters for local usability") and revises the boundary: interactive, verified-selected tools move the local model's *effective* ceiling, even though more parameters / more data are still what move the *base* capability.

Addendum 2 (update): the full inference-time stack, and what does not work

Pushing further on the local 4-bit 8B (BIRD text-to-SQL, held-out n=80, zero salary, all measured identically), a clear picture emerged: **inference-time scaffolding is the lever; weight-update is not.**

```
LOCAL 4-bit 8B, held-out n=80, execution accuracy

one-shot                          ███████████████░░░░░░░░░░░░░░░░░░░░░░░░░   37%
+ interactive tools (run/fix)     ███████████████████░░░░░░░░░░░░░░░░░░░░░   48%
+ in-context decomposition        █████████████████████░░░░░░░░░░░░░░░░░░░   52%
+ embedding-retrieved few-shot    █████████████████████░░░░░░░░░░░░░░░░░░░   53%   best
combined (all three)              █████████████████████░░░░░░░░░░░░░░░░░░░   52%   saturates
frontier baseline (Claude)        ██████████████████████████████░░░░░░░░░░   75%
```

What helped (each an inference-time lever): an interactive tool loop (run the SQL, read the error, fix); in-context decomposition (plan-then-solve); and few-shot examples retrieved by embedding similarity. Net lift from scaffolding alone: **37% to about 53%, +16 points, zero training and zero API cost.**

What did NOT help, and the lessons:

- **Weight-update at local data scale hurts.** Per-database test-time fine-tuning (LoRA on ~15 solved

examples) dropped accuracy to 32% (overfit and forgot); distillation on ~100 correct traces was flat. Small fine-tuning does not generalize on a heterogeneous task; that needs roughly 900k-scale data or RL.

- **The levers overlap, they do not stack.** Combining all three lands at the best single lever (~53%), not higher: they fix the same easier queries; the hard ~47% resist all of them (a base generation-capability limit, not a scaffolding limit).

- **Retrieval quality matters more than the idea.** Cheap lexical retrieval was flat (48%); embedding

retrieval reached 53%. Measure with the strong implementation before concluding a lever fails.

- **Unverified self-toolmaking backfires.** Letting the model invent its own pre-joined views (kept if they

merely ran) dropped accuracy to 22%; even frontier-designed correct views (41%) underperformed no views. The discipline that matters is **verified-selection**: keep a tool only if it raises end-task accuracy.

Practical upshot, unchanged and reinforced: for a local model, spend on the agent loop, a free verifier, good retrieval, and decomposition before spending on a bigger model or fine-tuning. The remaining gap to frontier is base capability, which needs scale or RL (out of the local-cheap scope).

Addendum 3 (final): the unified lever-map for local self-improvement

Across many methods on a 4-bit 8B (zero-salary, local), two levers separate cleanly:

- **Inference-time scaffolding works and is robust.** An agent loop (run + read error + fix) plus

decomposition plus embedding-retrieved examples lifts a hard task from 37% to ~53%, with no training and no forgetting. Verified-selection (keep a tool only if it raises end-task accuracy) is the discipline that makes it reliable; piling on tools or letting the model invent unverified ones *lowers* accuracy.

- **Weight-update self-improvement is fragile locally.** Distillation, test-time fine-tuning, and iterated

reward-gated fine-tuning all either stayed flat or *regressed* (catastrophic forgetting from narrow fine-tuning) — even on reachable tasks, even with gentle settings. It compounds only in a carefully engineered regime (a teacher plus heavy replay to prevent forgetting, on a homogeneous skill).

- **The hard boundary is reachability.** No method produces a correct answer the model can't already reach in

its sampling distribution; generation-bound tasks cap at that ceiling. Raising the ceiling itself needs scale or RL beyond a local laptop.

Practical takeaway, reinforced: for a local model, **invest in the agent loop + a free verifier + good retrieval before fine-tuning** — fine-tuning is the fragile, easy-to-regress lever, while scaffolding is the robust one.

Addendum 4: cross-task generalization and cost-optimal routing

The harness was extended to a generic agent loop (solve → execute → observe failure → fix) and run across seven verifiable task types through one interface. Two findings:

The scaffolding payoff is inverse to base accuracy (the "headroom law"). The agent loop and decomposition both help a lot where the model is weak but its errors are fixable, and almost nothing where it is already strong — across tasks, within a task, and across levers:

Task (verifier)	One-shot	Agent-loop	Decompose
SQL / BIRD (execution)	37%	48% (+11)	45% (+7)
Code / MBPP (unit tests)	70%	71% (+1)	69% (–1)
Code / HumanEval (unit tests)	84%	86% (+2)	—
Reasoning / GSM8K (exact)	~100%	saturated	—
Structured extraction / JSON (parse)	~100%	saturated	—

A 4-bit 8B is already strong on clean structured, reasoning, and code tasks; scaffolding earns its cost only on genuinely weak domains (heterogeneous multi-table SQL, spatial generation). Profile one-shot base first.

Cost-optimal routing (the practical accuracy/cost frontier). With a *correctness* verifier (unit tests at inference), the local model self-certifies — accept any answer that passes the tests (provably correct, no gold, no frontier call) and escalate only the rest:

Task	Self-certified locally (free)	Escalated	Routed accuracy (frontier≈0.75)	Frontier cost
MBPP	71%	29%	93%	29%
HumanEval	88%	12%	97%	12%

The verifier is the moat: acceptance is safe by construction (accepted answers pass the tests, so a wrong answer is never accepted). You reach near-frontier accuracy while paying the frontier for only a small residual. That is the build-vs-buy decision with numbers: local + verifier for the verifiable majority, escalate the tail.

Addendum 5: what the agent loop actually is (mechanism, fully reduced)

A series of ablations reduced the "agent loop" to a simple object, with a practical deployment rule.

It's retrying, not the feedback. Replacing the rich execution feedback ("this test failed: <trace>") with a generic "that was wrong, try again" changed nothing — on SQL, retry-only gained +11 while rich feedback gained +8 (the verbose error slightly *hurt* a small model). The lever is the verifier's accept/reject signal that gates a retry, not the error prose.

It's verified resampling, bounded by a measurable ceiling. The gain a model can get from *any* inference lever is bounded by its **reachable headroom** = pass@k oracle – one-shot (a few temperature samples + the verifier). Measured across four tasks (n=40), realized gain ≤ reachable headroom in every case, and raw headroom (100 – one-shot) is *not* predictive:

Task	one-shot	pass@8	reachable headroom	realized lever
SQL/BIRD	32%	~52%	+20	+11
ASCII (spatial)	50%	55%	+5	+0
MBPP (code)	78%	90%	+12	+1..+2
HumanEval	85%	88%	+2	+2

ASCII has lots of raw headroom but ~none reachable (its errors are systematic, not resample-fixable) — so no lever helps; it's a capability ceiling, route or scale instead.

Sequential retry ≠ best-of-N. Sequential retries are *correlated* (the model repeats similar errors on a hard item). On SQL that's fine (its correct answers have high per-sample probability — retry@2 captures most). On MBPP it isn't: retry saturates at +2 even at 8 rounds, while verifier-selected best-of-8 (independent samples) reaches +12. So for low-probability reachable headroom, use **independent best-of-N selected by the verifier**, not sequential retry — same verifier, much better coverage.

Deployment rule. Take ~8 temperature samples + verify. The spread (oracle – one-shot) is your achievable inference gain. If a 2-round retry already captures it, ship that (cheapest). If the spread is large but retry saturates below it, switch to verifier-selected best-of-N. If the spread is ~0, no inference lever will help — route the hard items to a stronger model, or scale. The verifier is the one component that makes all of this measurable and safe.

Addendum 6: the deployable local tier (putting it together)

Composing the validated pieces into one pipeline — one-shot → retry@2 → verifier-selected best-of-N → escalate the residual — and accepting any output that passes the task's own tests (provably correct, no gold, no frontier call). Measured on a 4-bit 8B, n=80:

Task	one-shot	local tier (self-certified)	cost (gens/item)	escalated
MBPP	70%	75%	2.9	25%
HumanEval	84%	90%	2.0	10%

The accuracy lift over one-shot is modest — reachable headroom is hard to realize, and sequential retry stalls (best-of-N's independent samples do the residual work: HumanEval +5 from best-of-N, +0 from retry). The point of the pipeline is not a big accuracy jump; it is **safe self-certification plus cost-routing**: the local model certifies the verifiable majority as *guaranteed correct* (a wrong answer can't pass the tests, so it's never accepted) at two-to-three generations per item, and flags a clean 10–25% tail to escalate to a stronger model. That is the deployable accuracy/cost/safety frontier for a local model with a correctness verifier — and it requires one (an execution-only "it ran" signal can't self-certify).